SE_Lab4

Last updated: 2026-10-15

User:

fix the inverted overlap<=0 check in drop_block()

Claude:

The fix is a one-character change: the success condition should be `overlap > 0`, so the drop only succeeds when the active block actually lands on the tower.

```
is_successful_drop = overlap &gt; 0
```

In context:

```
left = max(act.x, top_block.x)
right = min(act.x + act.width, top_block.x + top_block.width)
overlap = right - left

is_successful_drop = overlap &gt; 0

if is_successful_drop:
    trimmed_width = max(10.0, overlap)
    ...
```

With the old `overlap <= 0`, a miss (no horizontal overlap) counted as a successful stack and a real hit ended the game. Now a hit trims the block to the overlapping width and a miss triggers game over.

One related thing you may notice: the `max(10.0, overlap)` floor means a barely-overlapping drop still produces a 10px block, but `left` is the start of the overlap, so the block can extend past the edge of the tower below it when overlap is under 10px. If you want that to look cleaner, you could instead end the game when overlap falls below a small threshold (e.g. `overlap >= 10`).

Also, the restart hint says "Press [Space] or [R]", but `handle_event` restarts on R or left-click, not Space. If you want Space to work, add `event.key in (pygame.K_r, pygame.K_SPACE)` in the game-over branch, or change the text to "[R] or Click".

User:

make it so that snap within 3 px, show a “PERFECT!” popup, give bonus score, and widen the block after 3 perfects in a row.

Claude:

I've updated `game_engine.py` with the overlap fix and the perfect-drop features.

- **Snap within 3 px:** if the active block's x is within 3 px of the block below, the new block snaps to its x and keeps its full width, so nothing is trimmed.
- **"PERFECT!" popup:** a text popup floats up from the new block and fades out over about 45 frames.
- **Bonus score:** a perfect drop gives +3 (the normal +1 plus a +2 bonus).
- **Widening:** after 3 perfects in a row, the block grows by 20 px, evenly on both sides, and the popup reads "PERFECT! WIDER!". The streak then resets, so the next widen needs another 3 perfects. Any non-perfect drop also resets the streak.

The tuning values are constants at the top of the file (`PERFECT_TOLERANCE`, `PERFECT_BONUS`, `PERFECT_STREAK_WIDEN`, `WIDEN_AMOUNT`).

I also made some changes you didn't ask for:

- **Width cap:** widening is capped at `base_width` (180 px), so the tower can't grow wider than where it started. Raise or remove the cap if you want it uncapped.
- **HUD label:** since bonus points make the score no longer equal the tower height, I renamed "Height" to "Score" on the HUD and in "Final Score" on the game-over screen.
- **Restart hint:** it now reads "Press [R] or Click to Play Again", matching what `handle_event` actually accepts.

User:

make it so that when a block is trimmed, make the sliced-off piece fall with gravity and rotation until it leaves the screen just give me the part where i should change it.

Claude:

All changes are in `game_engine.py`.

1. Add a gravity constant at the top of the file

```
GRAVITY = 0.6  # downward acceleration per frame for sliced-off pieces
```

2. In `reset()`, add this line next to `self.popups = []`

```
self.falling_pieces = []
```

3. Add this new method (e.g. right after `add_popup`)

```
def spawn_falling_piece(self, x, y, width, color, direction):
    # direction: -1 if the piece was cut off the left side, +1 for the right
    self.falling_pieces.append({
        "x": x, "y": y, "w": width, "h": self.block_height,
        "color": color,
```

```
            "vx": direction * random.uniform(0.5, 1.5),
            "vy": 0.0,
            "angle": 0.0,
            "spin": direction * random.uniform(-4.0, -2.0),
        })
```

4. In `drop_block()`, change the non-perfect `else:` branch

```
    else:
        new_x = left
        new_width = max(10.0, overlap)
        self.perfect_streak = 0
        popup_text = None

        # slice off the overhang and let it fall
        if act.x &lt; top_block.x:
            cut_w = left - act.x
            if cut_w &gt; 0.5:
                self.spawn_falling_piece(act.x, act.y, cut_w, act.color, -1)
        else:
            cut_w = (act.x + act.width) - right
            if cut_w &gt; 0.5:
                self.spawn_falling_piece(right, act.y, cut_w, act.color, 1)
```

5. In `update()`, add this after the popup loop

```
    for p in self.falling_pieces:
        p["vy"] += GRAVITY
        p["x"] += p["vx"]
        p["y"] += p["vy"]
        p["angle"] += p["spin"]
    self.falling_pieces = [p for p in self.falling_pieces if p["y"] &lt;
    self.height + 100]
```

6. In `render()`, add this right before the `for p in self.popups:` loop

```
    for p in self.falling_pieces:
        w, h = int(p["w"]), int(p["h"])
        piece = pygame.Surface((w, h), pygame.SRCALPHA)
        pygame.draw.rect(piece, p["color"], (0, 0, w, h), border_radius=4)
        pygame.draw.rect(piece, (245, 245, 250), (0, 0, w, h), width=2,
    border_radius=4)
        rotated = pygame.transform.rotate(piece, p["angle"])
        center = (p["x"] + p["w"] / 2, p["y"] + p["h"] / 2)
        screen.blit(rotated, rotated.get_rect(center=center))
```

The piece rotates around its own center, drifts outward away from the tower, and is removed once it falls 100 px below the screen.

If you also want the whole block to fall when you miss completely (game over), add this inside the `if not is_successful_drop:` branch, before `self.game_over = True`:

```
self.spawn_falling_piece(act.x, act.y, act.width, act.color, 1 if
act.direction &gt; 0 else -1)
```

User:

Also as the score rises, gradually fade the background through twilight blue, dusk purple, a night starfield and stratosphere black.

Claude:

All changes are in `game_engine.py`.

1. Add this near the top of the file, with the other constants

```
# (score, RGB) keyframes - the sky fades smoothly between them
SKY_STAGES = [
    (0,  (24, 27, 36)),    # starting dark slate
    (8,  (38, 78, 140)),   # twilight blue
    (20, (82, 48, 128)),   # dusk purple
    (35, (10, 12, 36)),    # night
    (55, (0, 0, 0)),       # stratosphere black
]
STAR_FADE_START = 22   # score where stars begin to appear
STAR_FADE_END = 36     # score where stars are fully visible


def lerp_color(a, b, t):
    return tuple(int(a[i] + (b[i] - a[i]) * t) for i in range(3))
```

2. In `__init__()`, add this before `self.reset()`

```
self.stars = [
    (random.randint(0, width), random.randint(0, height),
     random.choice([1, 1, 2]), random.uniform(0, 6.28))
    for _ in range(90)
]
```

3. In `reset()`, add this next to `self.popups = []`

```
self.bg_t = 0.0  # smoothed score used for the sky, so bonus jumps don't pop
```

4. Add these two methods (e.g. after `add_popup`)

```
    def sky_color(self, s):
        if s &lt;= SKY_STAGES[0][0]:
            return SKY_STAGES[0][1]
        for (s0, c0), (s1, c1) in zip(SKY_STAGES, SKY_STAGES[1:]):
            if s &lt;= s1:
                return lerp_color(c0, c1, (s - s0) / (s1 - s0))
        return SKY_STAGES[-1][1]

    def draw_stars(self, screen):
        strength = (self.bg_t - STAR_FADE_START) / (STAR_FADE_END -
    STAR_FADE_START)
        strength = max(0.0, min(1.0, strength))
        if strength &lt;= 0:
            return
        t = pygame.time.get_ticks() / 1000.0
        for x, y, r, phase in self.stars:
            twinkle = 0.65 + 0.35 * math.sin(t * 2.0 + phase)
            c = int(255 * strength * twinkle)
            pygame.draw.circle(screen, (c, c, c), (x, y), r)
```

This uses `math`, so add `import math` at the top of the file.

5. In `update()`, add this at the start of the method

```
    self.bg_t += (self.score - self.bg_t) * 0.05
```

6. In `render()`, replace the first line

```
    screen.fill((24, 27, 36))
```

with:

```
    screen.fill(self.sky_color(self.bg_t))
    self.draw_stars(screen)
```

The stars are drawn before the blocks, so they sit behind the tower.

The stage thresholds are in `SKY_STAGES`, so you can stretch or compress the progression there. Because bonus points count toward the score, a streak of perfects moves the sky along faster than the tower height alone.

---

*This document was created using **Claudify** ⚡ – Your smart solution for turning chats into PDFs.*
Download Claudify now to preserve and organize your conversations effortlessly!